

데이터 구조 설계

3차 과제 보고서

학번 : 2022202109

이름 : 이호정

Introduction

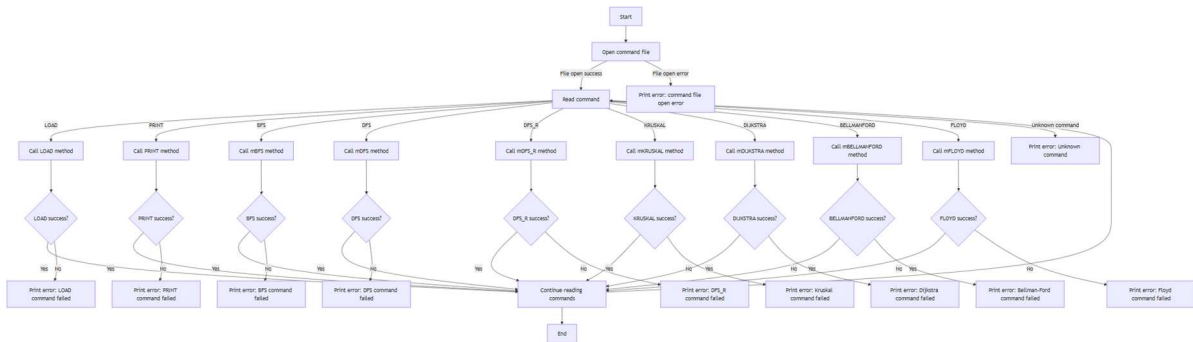
이 프로젝트는 지금까지 배운 알고리즘과 데이터구조들을 이용해서 그래프 연산 프로그램을 구현한다.

본 프로젝트는 다양한 그래프 데이터 구조(ListGraph 및 MatrixGraph)를 활용하여 그래프 기반의 알고리즘을 구현하고 이를 명령어 기반으로 실행할 수 있는 프로그램을 개발하는 것을 목표로 한다. 주요 알고리즘으로는 BFS(너비 우선 탐색), DFS(깊이 우선 탐색), Kruskal(최소 신장 트리), Dijkstra(단일 시작점 최단 경로), Bellman-Ford(음수 가중치 포함 최단 경로), 그리고 Floyd-Warshall(모든 쌍 최단 경로)가 포함되어 있다. 주요 기능으로는 그래프 데이터를 파일로부터 불러오는 LOAD 명령어, 그래프의 현재 상태를 출력하는 PRINT 명령어, 특정 시작점에서 BFS 및 DFS를 수행하는 탐색 기능이 포함됩니다. 또한, Kruskal 알고리즘으로 최소 신장 트리를 생성하고, Dijkstra 알고리즘으로 단일 시작점 최단 경로를 계산하며, Bellman-Ford 알고리즘으로 음수 가중치 포함 최단 경로를 계산합니다. 모든 정점 간 최단 경로를 구하는 Floyd-Warshall 알고리즘도 제공됩니다. 명령어가 잘못되거나 그래프 데이터가 없는 경우 적절한 오류 메시지를 출력하도록한다.

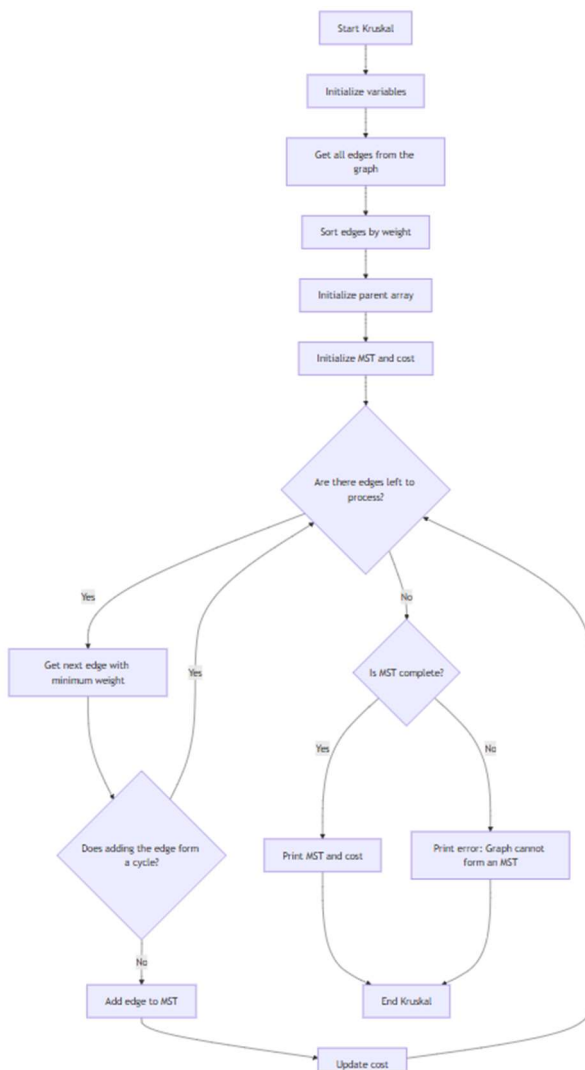
각 그래프 알고리즘은 그래프 데이터 구조의 특성을 최대한 활용하도록 설계되었습니다. ListGraph는 공간 효율적이고 동적 데이터에 적합하며, MatrixGraph는 정적이고 밀집된 그래프에 적합합니다. 프로그램은 이러한 데이터 구조와 알고리즘을 결합하여 그래프 문제를 효율적으로 해결할 수 있는 유연성을 제공합니다. 본 프로젝트는 그래프 구조와 알고리즘을 학습하는 데 유용하며, 향후 그래프 기반 문제를 해결하거나 최적화 알고리즘을 연구하는 데 활용될 수 있다.

Flowchart

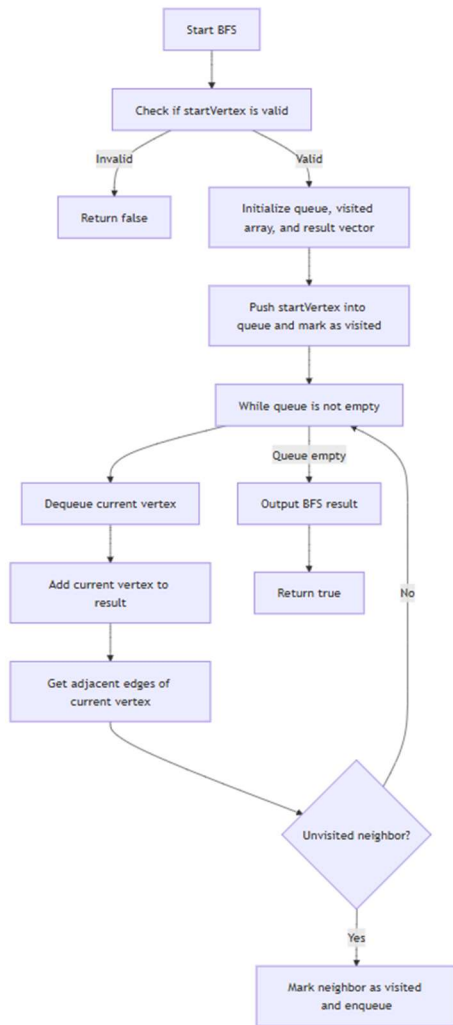
전체적인 흐름



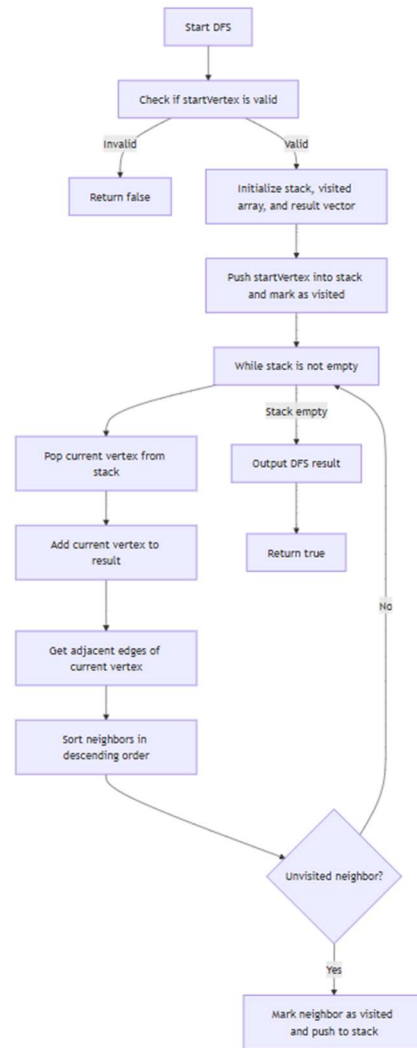
KRUSKAL 플로우차트



BFS 플로우차트



DFS 플로우차트



Algorithm

이 프로젝트에서 사용된 알고리즘은 그래프 이론의 기초적인 개념을 바탕으로 다양한 문제를 해결하는 데 초점을 맞추고 있다. 각 알고리즘은 그래프의 특정 속성을 탐색하거나 최적화 문제를 해결하는 데 사용되며, 아래에서 그 동작 원리를 설명한다.

BFS (Breadth-First Search, 너비 우선 탐색)은 그래프의 모든 정점을 방문하기 위해 사용된다. 시작 정점을 큐에 삽입하고, 해당 정점의 인접한 모든 정점을 탐색하며 큐에 추가한다. 이 과정은 각 정점을 방문하며 탐색이 완료될 때까지 반복된다. BFS는 최단 경로를 탐색하거나 그래프의 연결성을 확인하는 데 유용하다.

DFS (Depth-First Search, 깊이 우선 탐색)은 그래프를 깊이 있게 탐색하는 알고리즘이다. 시작 정점에서 출발해 가능한 깊이까지 내려가며 정점을 방문하고, 더 이상 방문할 정점이 없으면 되돌아오는 방식으로 동작한다. 이 과정은 스택을 이용해 구현되며, 특정 경로 탐색이나 순환 여부를 확인하는 데 적합하다.

DFS_R은 DFS의 재귀 구현 버전이다. 재귀 호출을 통해 그래프를 탐색하며, 방문하지 않은 정점을 재귀적으로 호출하면서 깊이 우선 탐색을 수행한다. 비재귀 방식과 비교해 구현은 간결하지만, 호출 스택 크기에 주의해야 한다.

Kruskal 알고리즘은 최소 신장 트리(MST)를 찾는 알고리즘 중 하나로, 모든 간선을 가중치에 따라 정렬한 뒤, 가중치가 낮은 간선부터 추가한다. 이때, 사이클이 형성되지 않도록 유니온-파인드(Union-Find) 자료구조를 사용해 각 정점의 부모를 관리한다. 그래프의 연결성을 유지하면서 비용을 최소화할 수 있다.

Dijkstra 알고리즘은 시작 정점에서 다른 모든 정점으로의 최단 경로를 구하는 알고리즘이다. 각 정점의 거리를 초기화한 뒤, 방문하지 않은 정점 중 거리가 가장 짧은 정점을 선택하고, 그 정점과 인접한 정점들의 거리를 갱신한다. 이 과정은 모든 정점이 처리될 때까지 반복되며, 음수 가중치를 처리하지 못하는 한계가 있다.

Bellman-Ford 알고리즘은 음수 가중치를 포함한 최단 경로를 구할 수 있는 알고리즘이다. 모든 간선을 반복적으로 탐색하며 최단 경로를 갱신하고, 이 과정을 (정점의 개수 - 1) 번 반복한다. 마지막으로 음수 사이클이 존재하는지 확인하여, 사이클이 감지되면 오류를 출력한다.

Floyd-Warshall 알고리즘은 그래프의 모든 정점 쌍 사이의 최단 경로를 구하는 알고리즘이다. 동적 계획법(Dynamic Programming)을 사용하여 중간 정점을 기준으로 경로를

갱신하며, 음수 가중치를 처리할 수 있다. 그러나 음수 사이클이 있는 경우 이를 감지하고 경고를 출력한다.

이러한 알고리즘들은 각각의 그래프 문제를 해결하는 데 최적화되어 있으며, 다양한 활용 사례에 적용될 수 있다. 프로젝트에서는 이 알고리즘들을 통해 그래프의 구조를 분석하고, 사용자로부터 입력받은 명령어에 따라 원하는 결과를 출력하는 데 성공했다.w

Result Screen

```
LOAD graph_M.txt
PRINT
LOAD graph_L.txt
PRINT
BFS 0
DFS 2
DFS_R 2
KRUSKAL
DIJKSTRA 5
BELLMANFORD 0 6
FLOYD
EXIT
```

```
L
7
0 1 6 2 2
1 3 5
2 1 7 4 3 5 8
3 6 3
4 3 4
5 6 1
6 4 10
```

```
M
7
0 6 2 0 0 0 0
0 0 0 5 0 0 0
0 7 0 0 3 8 0
0 0 0 0 0 0 3
0 0 0 4 0 0 0
0 0 0 0 0 0 1
0 0 0 0 10 0 0
```

➔ Command.txt, graph_M.txt, graph_L.txt 가 이렇게 입력 데이터로 주어졌을 때

LOAD graph_M.txt 와 PRINT 명령어가 올바르게 수행된 것을 확인할 수 있다.

```
===== LOAD =====
Success
=====

=====PRINT=====
      [0]  [1]  [2]  [3]  [4]  [5]  [6]
[0]    0    6    2    0    0    0    0
[1]    0    0    0    5    0    0    0
[2]    0    7    0    0    3    8    0
[3]    0    0    0    0    0    0    3
[4]    0    0    0    4    0    0    0
[5]    0    0    0    0    0    0    1
[6]    0    0    0    0    10   0    0
=====
```

LOAD graph_L.txt 와 PRINT 명령어가 올바르게 수행된 것을 확인할 수 있다.

```
===== LOAD =====  
Success  
=====
```

```
=====PRINT=====
```

```
[0] -> (1,6) -> (2,2)  
[1] -> (3,5)  
[2] -> (1,7) -> (4,3) -> (5,8)  
[3] -> (6,3)  
[4] -> (3,4)  
[5] -> (6,1)  
[6] -> (4,10)  
=====
```

```
===== BFS =====  
startvertex: 0  
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6  
=====
```

```
===== DFS =====  
startvertex: 2  
2 -> 0 -> 1 -> 3 -> 6 -> 4 -> 5  
=====
```

```
===== DFS_R=====
```

```
startvertex: 2  
2 -> 0 -> 1 -> 3 -> 4 -> 6 -> 5  
=====
```

➔ 각각 BFS, DFS, DSF_R 이 모두 올바르게 실행된 것을 확인할 수 있다. 모두 0을 제외하지 않고 포함하였으며 DFS 는 스택으로 DSF_R 은 재귀적으로 구현하였다.

```

===== Kruskal =====
[0] 2(2)
[1] 3(5)
[2] 0(2) 4(3)
[3] 6(3) 4(4) 1(5)
[4] 2(3) 3(4)
[5] 6(1)
[6] 5(1) 3(3)
cost: 18
=====

```

→ KRUSKAL 명령어가 모두 올바르게 실행된 것을 확인할 수 있다.

```

===== Dijkstra =====
startvertex: 5
[0] 5 -> 2 -> 0 (10)
[1] 5 -> 6 -> 3 -> 1 (9)
[2] 5 -> 2 (8)
[3] 5 -> 6 -> 3 (4)
[4] 5 -> 6 -> 3 -> 4 (8)
[5] 5 (0)
[6] 5 -> 6 (1)
=====
----- Bellman-Ford -----

```

→ DIJKSTRA 명령어가 모두 올바르게 실행된 것을 확인할 수 있다.

```

-----
===== Bellman-Ford =====
0 -> 2 -> 5 -> 6
cost: 11
=====

```

→ BELLMANFORD 명령어가 모두 올바르게 실행된 것을 확인할 수 있다.


```

=====
===== FLOYD =====
      [0]  [1]  [2]  [3]  [4]  [5]  [6]
[0]   0   6   2   9   5  10  11
[1]   6   0   7   5   9   9   8
[2]   2   7   0   7   3   8   9
[3]   9   5   7   0   4   4   3
[4]   5   9   3   4   0   8   7
[5]  10   9   8   4   8   0   1
[6]  11   8   9   3   7   1   0
=====
=====

```

➔ FLOYD 명령어가 모두 올바르게 실행된 것을 확인할 수 있다.

Consideration

이 프로그램을 개발하면서 그래프 이론과 알고리즘에 대해 깊이 있는 이해를 할 수 있었다. 특히 BFS, DFS, Dijkstra, Bellman-Ford, 그리고 Floyd-Warshall과 같은 다양한 알고리즘을 직접 구현하면서, 각각의 알고리즘이 어떤 문제를 해결하기 위해 설계되었는지 더 명확히 알게 되었다. 단순히 이론적으로만 배울 때는 잘 이해가 되지 않았던 부분들도 실제로 코드로 작성하고, 디버깅하면서 그 의미와 동작 방식을 체감할 수 있었다. 특히, 다양한 데이터 구조를 사용하여 그래프를 구현하는 과정이 흥미로웠다. ListGraph와 MatrixGraph의 차이를 구현하고 비교하면서, 데이터 구조의 선택이 성능에 얼마나 큰 영향을 미치는지 알게 되었다. 또한, 어떤 상황에서는 연결 리스트 기반의 그래프가 더 적합하고, 어떤 경우에는 인접 행렬이 효율적이라는 점도 깨달았다. 이러한 선택이 실제 프로그램의 동작과 효율성에 미치는 영향을 이해할 수 있는 기회였다. 또한, 그래프 알고리즘을 구현하는 과정에서 예상하지 못한 다양한 오류와 버그를 만나면서 디버깅 능력도 향상되었다. 처음에는 단순히 알고리즘을 구현하는 데 집중했지만, 점점 더 예외 상황에 대한 처리를 고려하게 되었다. 예를 들어, 음수 가중치나 음수 사이클과 같은 예외적인 상황은 단순히 알고리즘을 구현하는 것 이상의 고민을 요구했다. 이를 해결하면서 문제 해결 능력이 조금은 늘어난 것 같다는 생각이 들었다. 이 프로젝트를 하면서 협업의 중요성도 느꼈다. 친구의 알고리즘을 참고하면서, 서로 다른 구현 방식이 어떤 장단점을 가지고 있는지 비교해보는 과정이 유익했다. 혼자서만 개발했다면 느끼지 못했을 다양한 관

점과 접근 방식을 배울 수 있었다. 동시에, 다른 사람의 코드를 읽고 이해하는 능력도 필요하다는 점을 깨달았다. 마지막으로, 실제로 동작하는 프로그램을 완성하는 것이 얼마나 뿌듯한 일인지 느낄 수 있었다. 비록 오류와 버그를 해결하는 과정이 반복되어 힘들기도 했지만, 하나씩 문제를 해결해 나가면서 점점 프로그램이 완성되는 모습을 보는 것은 성취감을 주었다. 컴퓨터공학과 2학년으로서, 이런 프로젝트를 통해 실질적인 프로그래밍 경험을 쌓을 수 있었고, 이 경험이 앞으로의 학업과 진로에 큰 도움이 될 것이라는 생각이 든다.